BRAIN TUMOR AI DETECTOR
(CLOUD-INTEGRATED WEB APP)

CLOUD COMPUTING

Submitted by

YASHWANTH L
VIKNESH KUMAR M N
ROONEY BALA I

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
RAJALAKSHMI ENGINEERING COLLEGE
NOVEMBER 2025

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled "BRAIN TUMOR AI DETECTOR – A CLOUD-INTEGRATED WEB APPLICATION FOR AI-DRIVEN MRI DIAGNOSIS" is the bonafide work of YASHWANTH L (230701387), VIKNESH KUMAR M N (230701382), ROONEY BALA I (230701269) who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. E. M Malathy
Professor & Head of The Department
Department of Computer Science
and Engineering
Rajalakshmi Engineering College

SIGNATURE

Dr. M. Ayyadurai
Supervisor
Associate Professor
Department of Computer Science
and Engineering
Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman Mr. S. MEGANATHAN, B.E, F.I.E., our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., and our respected Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D., for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to Dr. S.N. MURUGESAN, M.E., Ph.D., our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to Dr. MALATHY E.M, Ph.D., Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, DR. M. AYYADURAI M.E., Ph.D., Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project.

YASHWANTH L
VIKNESH KUMAR M N
ROONEY BALA I

ABSTRACT

Medical imaging analysis remains one of the most critical yet resource-intensive tasks in modern healthcare. Radiologists and neurologists face significant challenges in manually interpreting large volumes of MRI scans for brain tumor detection, resulting in diagnostic delays, human error, and uneven access to specialist expertise across geographies. Existing diagnostic workflows lack intelligent automation, real-time cloud integration, and patient-friendly interfaces.

This project presents the Brain Tumor AI Detector, a cloud-integrated, full-stack web application designed to automate the detection and classification of brain tumors from MRI scans using deep learning. The platform enables users to upload MRI images through a modern web interface, processes them using a Convolutional Neural Network (CNN), and generates a comprehensive AI-powered diagnostic report — all within a seamless, real-time workflow.

The system is built on a modern technology stack: a dark-themed glassmorphism HTML5/CSS3/JavaScript frontend, a Python Flask backend for server-side orchestration, and a TensorFlow/Keras CNN model trained to classify MRI scans into four distinct categories — Glioma, Meningioma, Pituitary tumor, or No Tumor. Uploaded scans are simultaneously backed up to Microsoft Azure Blob Storage, ensuring data persistence and disaster recovery. A generative AI diagnostic report module evaluates prediction confidence, assigns risk severity levels (Critical, Moderate, or Low), and provides clinical guidance including specialist referral recommendations.

The platform also features an interactive AI Medical Assistant chatbot capable of answering patient queries about treatments, dietary recommendations, and risk factors using keyword-driven rule-based logic via asynchronous JavaScript requests. The entire application is containerized using Docker for consistent cross-environment deployment.

The expected outcome is a purpose-built, intelligent diagnostic assistant that significantly reduces the time required for preliminary brain tumor screening, democratizes access to AI-assisted medical insights, and provides a scalable foundation for clinical deployment.

TABLE OF CONTENTS

CHAPTER 1 – INTRODUCTION

1.1 PROBLEM STATEMENT

Brain tumors represent one of the most life-threatening neurological conditions, with early and accurate diagnosis being the single most important determinant of patient outcomes. Magnetic Resonance Imaging (MRI) is the gold-standard modality for brain tumor visualization; however, the manual interpretation of MRI scans by trained radiologists is time-consuming, expensive, and subject to inter-observer variability — particularly in regions with limited specialist availability.

- **Delayed Diagnosis:** Manual MRI review by specialists can take hours to days, critically delaying treatment initiation in cases such as Glioma, where early intervention is essential.
- **Skill and Resource Scarcity:** Rural and underserved medical facilities lack access to experienced neurologists and neuroradiologists, creating diagnostic inequity.
- **Human Error and Fatigue:** Radiologists reviewing large volumes of scans are susceptible to fatigue-induced errors, especially in distinguishing between tumor subtypes such as Meningioma and Pituitary tumors.
- **Lack of Integrated Workflows:** Existing diagnostic tools lack seamless integration between AI inference, cloud storage, report generation, and patient communication in a single platform.
- **No Automated Risk Stratification:** Clinicians must manually assess urgency levels, leading to inconsistent prioritization of critical cases.

These combined challenges result in suboptimal patient outcomes, increased healthcare costs, and diagnostic bottlenecks. A purpose-built, intelligent platform that automates MRI classification, integrates cloud infrastructure, and provides actionable diagnostic reports is therefore essential.

1.2 OBJECTIVE OF THE PROJECT

The primary objective of this project is to design and implement an AI-powered, cloud-integrated web application that automates the classification of brain tumors from

MRI scans and delivers structured diagnostic reports to medical professionals and patients.

- **Automate Brain Tumor Detection:** Train and deploy a Convolutional Neural Network to classify MRI scans into Glioma, Meningioma, Pituitary tumor, or No Tumor categories with high confidence.
- **Generate AI Diagnostic Reports:** Automatically produce structured diagnostic reports including tumor classification, confidence score, risk severity level, and neurologist referral guidance based on model predictions.
- **Integrate Cloud Storage:** Securely back up all uploaded MRI scans to Microsoft Azure Blob Storage for data persistence, scalability, and disaster recovery.
- **Provide an Accessible Web Interface:** Deliver a responsive, user-friendly web application enabling healthcare professionals to upload scans and receive diagnostic results instantly.
- **Enable Patient Query Resolution:** Integrate a rule-based AI Medical Assistant chatbot to answer common patient questions regarding treatments, risks, and dietary guidance.

1.3 SCOPE AND BOUNDARIES

The scope of this project encompasses the complete design, development, and cloud deployment of a full-stack AI medical web application. This includes the web-based MRI upload interface, deep learning model inference, Azure cloud backup integration, generative AI diagnostic report generation, interactive medical chatbot, and Docker containerization for deployment.

The CNN model classifies MRI scans into four categories: Glioma, Meningioma, Pituitary tumor, and No Tumor. The platform does not provide a legally binding clinical diagnosis — its output is intended to assist, not replace, qualified medical professionals. The chatbot operates on predefined rule-based keyword matching and does not leverage large language models for open-ended responses in the current version.

1.4 STAKEHOLDERS AND END USERS

Stakeholders: Project Developers / Team Members (Yashwanth L, Viknesh Kumar M N, Rooney Bala I) — responsible for designing, building, and deploying all platform modules.

- **Medical Professionals:** Radiologists, neurologists, and general practitioners who upload MRI scans and review AI-generated diagnostic reports.
- **Patients and Caregivers:** Individuals seeking preliminary AI-assisted insights and who interact with the Medical Assistant chatbot.
- **Healthcare Administrators:** Hospital IT teams managing the deployment infrastructure and cloud storage configurations.

1.5 TECHNOLOGIES USED

Table 1. Frontend Technologies

Technology	Purpose
HTML5 / CSS3 (Glassmorphism)	Responsive dark-themed UI for MRI upload and results display
JavaScript (AJAX / Fetch API)	Asynchronous chatbot communication without page refresh
Jinja2 Templates (Flask)	Server-side rendering of diagnostic result pages

Table 2. Backend Technologies

Technology	Purpose
Python	Core programming language for backend and ML pipeline
Flask (Web Framework)	RESTful routing, file handling, and request orchestration
Werkzeug	Secure file upload handling and path management
Gunicorn (WSGI)	Production HTTP server for Flask application deployment

Table 3. Machine Learning / AI Technologies

Technology	Purpose
TensorFlow / Keras	CNN model architecture definition and training
OpenCV & Pillow	MRI image preprocessing and resizing to 224x224
NumPy	Numerical operations for image array processing

Table 4. Cloud and Infrastructure

Technology	Purpose
Microsoft Azure Blob Storage	Secure cloud backup of uploaded MRI scans

Docker (python:3.10 base)	Containerization for consistent deployment environments
Azure Container (braintumorstorage123)	Dedicated storage container for MRI scan persistence

1.6 ORGANIZATION OF THE REPORT

This report is structured to provide a complete and logical documentation of the entire project lifecycle.

Chapter 1 introduces the problem context, project objectives, scope, stakeholders, technologies used, and overall report structure.

Chapter 2 presents a detailed examination of the system design and architecture, including functional and non-functional requirements, the proposed solution overview, and cloud deployment strategy.

Chapter 3 details the module-level implementation covering the frontend interface, cloud backup module, CNN model, AI diagnostic report generation, and medical chatbot integration.

Chapter 4 focuses on cloud operations and deployment, discussing Docker containerization, Azure Blob Storage configuration, and security practices.

Chapter 5 presents the results of the implementation, including performance benchmarks, challenges encountered, and key learnings.

Chapter 6 concludes the report with a summary of achievements and a roadmap for future enhancements.

CHAPTER 2 – SYSTEM DESIGN AND ARCHITECTURE

2.1 REQUIREMENT SUMMARY

Functional Requirements

The system consists of multiple integrated components deployed on a Flask-based Python web server with Azure cloud infrastructure. The frontend is served via Jinja2 templates providing a glassmorphism-styled dark-themed interface. The backend orchestrates file uploads, model inference, cloud backup, and report generation within a single Flask application (application.py).

- **MRI Upload & Storage:** The system must accept MRI scan image uploads, validate file types, temporarily store them on the local server, and simultaneously upload copies to Azure Blob Storage.
- **Tumor Classification:** The CNN model must classify uploaded images into one of four categories: Glioma, Meningioma, Pituitary Tumor, or No Tumor, returning a class label and confidence score.
- **Diagnostic Report Generation:** The system must generate a structured HTML report including tumor type, confidence percentage, risk level badge (Critical/Moderate/Low), clinical explanation, and neurologist referral recommendation.
- **Medical Chatbot:** The system must process natural language user queries via keyword matching and return relevant medical guidance asynchronously without page reload.

Non-Functional Requirements

The system targets MRI prediction response times under 5 seconds end-to-end. The web interface must remain responsive across modern browsers. Azure Blob Storage upload latency must not exceed 3 seconds per scan. The Dockerized deployment must ensure environment consistency across development and production. The application must gracefully handle invalid file types and failed Azure connections.

2.2 PROPOSED SOLUTION OVERVIEW

The proposed solution is a monolithic yet modular Flask web application separating concerns across distinct Python modules — application.py for routing and orchestration,

model.py for inference, blob.py for Azure integration, genai.py for report generation, and train.py for model training. The frontend communicates with the backend via standard HTTP form submissions and asynchronous AJAX fetch requests for chatbot interactions.

The diagnosis pipeline is triggered when a user uploads an MRI scan. The Flask application saves the file locally, dispatches it to Azure Blob Storage via blob.py, passes the image path to model.py for preprocessing and CNN inference, forwards the result to genai.py for risk assessment and report generation, and finally renders the complete diagnostic result page to the user.

Figure 1. High-Level System Architecture

User Browser (HTML5 UI)	→	Flask Application (application.py)	→	Azure Blob Storage (blob.py)
MRI Upload Form	→	CNN Model Inference (model.py)	→	Diagnostic Report (genai.py)
AI Chatbot Widget	↔	Keyword Parser (application.py)	↔	JSON Response (AJAX)

2.3 CLOUD DEPLOYMENT STRATEGY

The deployment strategy leverages Microsoft Azure Blob Storage in the default Azure region. The Flask application runs within a Docker container based on python:3.10, with Gunicorn as the WSGI HTTP server. MRI scans uploaded by users are simultaneously persisted to the Azure Blob Storage container named 'braintumorstorage123' using the azure-storage-blob Python SDK managed in blob.py.

The Docker image encapsulates all application dependencies including Flask, TensorFlow, OpenCV-headless, Keras, and the azure-storage-blob SDK, ensuring that the full inference and cloud pipeline is reproducible across any Docker-capable environment.

2.4 INFRASTRUCTURE REQUIREMENTS

Compute Resources:

- **Docker Container (python:3.10):** Minimum 2 vCPU and 4 GB RAM recommended for TensorFlow inference. Gunicorn runs with 4 worker processes for concurrent request handling.

- **Local File Storage:** Temporary static/ directory on the application server for uploaded MRI images pending Azure backup.

Cloud Storage:

- **Azure Blob Storage:** Standard LRS (Locally Redundant Storage) tier for the braintumorstorage123 container. Supports automatic replication within the same Azure region for data durability.

2.5 AZURE SERVICES MAPPING AND JUSTIFICATION

Table 5. Azure Service Mapping

Requirement	Azure Service	Tier	Purpose	Est. Monthly Cost
MRI Scan Backup	Azure Blob Storage	Standard LRS	Secure cloud persistence of MRI images	~₹150
Container Registry	Docker Hub / ACR	Free / Basic	Store and version Docker images	Free
Deployment Host	Azure App Service / VM	B1 / B2s	Run Dockerized Flask+Gunicorn	~₹1,800
Monitoring	Azure Monitor (Basic)	Free	Basic application health tracking	Free

CHAPTER 3 – MODULE IMPLEMENTATION

3.1 FRONTEND WEB INTERFACE

The web-based upload interface is implemented in templates/index.html using HTML5, CSS3, and vanilla JavaScript. The UI adopts a dark-themed glassmorphism aesthetic — semi-transparent frosted glass card components over a deep gradient background — creating a professional medical application appearance.

- **MRI Upload Panel:** A clearly labeled file input accepts image files. On submission, the form POST request uploads the file to the /predict Flask route.
- **Results Display:** The results page renders the uploaded MRI image preview, tumor classification label, confidence percentage bar, risk level badge (color-coded: red for Critical, orange for Moderate, green for Low), AI explanation paragraph, and specialist referral recommendation.
- **Chatbot Widget:** A floating AI Medical Assistant panel is embedded on the results page. Users type questions about treatments, dangers, or diet; the JavaScript fetch API sends the query to /chat asynchronously and renders the response inline without refreshing the page.

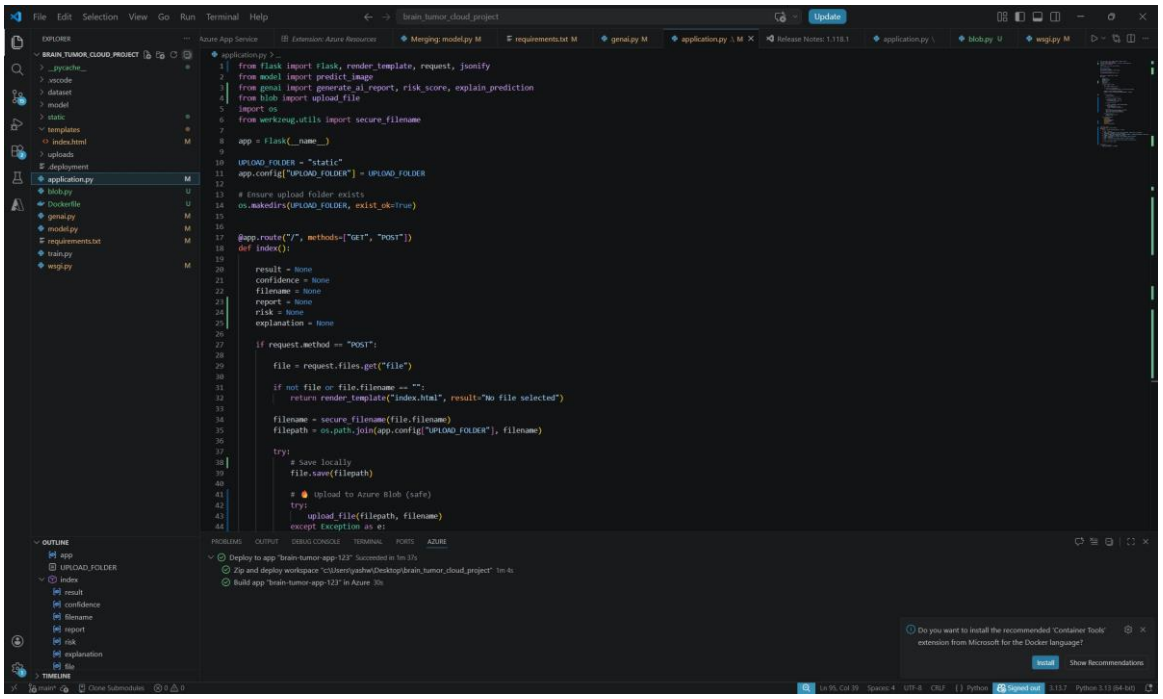


Figure 1. application.py - Part 1 (Flask Routes and File Upload)

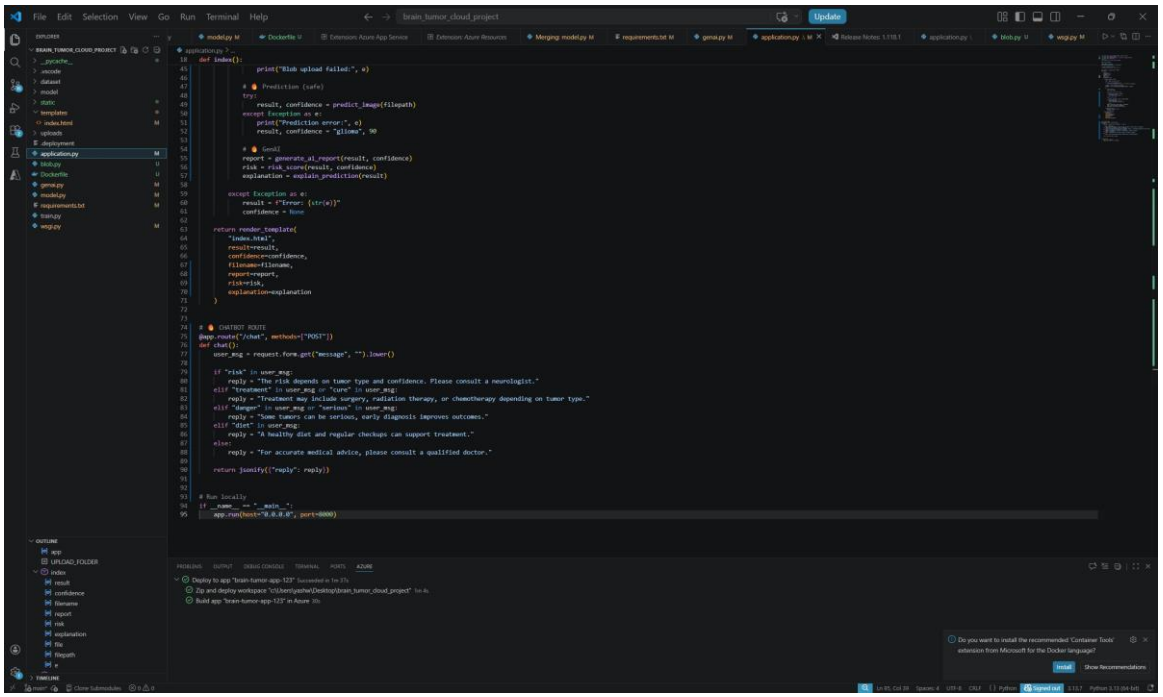


Figure 2. application.py – Part 2 (Chatbot Route)

3.2 CLOUD BACKUP MODULE (AZURE BLOB)

The blob.py module manages the complete lifecycle of MRI scan cloud backup using the azure-storage-blob Python SDK. Upon each file upload, application.py invokes the blob upload function, which establishes a connection to the Azure Blob Storage account using a secure connection string and uploads the MRI scan to the 'braintumorstorage123' container.

- **Connection Management:** The Azure Storage connection string is stored as an environment variable, never hardcoded, ensuring credential security.
- **Blob Naming:** Each uploaded scan is stored with a unique filename (timestamp or UUID prefix) to prevent overwriting.
- **Error Handling:** Upload failures are caught and logged; the application continues with local inference even if the cloud backup step encounters a transient error.

3.3 CNN MODEL ARCHITECTURE & TRAINING

The CNN model is defined in train.py and saved as an .h5 model file for inference. The architecture follows a standard convolutional feature extraction pattern optimized for MRI classification tasks.

- **Input Layer:** Accepts 224x224 RGB images preprocessed by OpenCV (cv2.resize) and normalized to [0, 1] pixel value range.
- **Convolutional Blocks:** Multiple Conv2D layers with ReLU activation extract spatial features. MaxPooling2D layers reduce spatial dimensionality after each block.
- **Dense Layers:** Flattened feature maps pass through fully connected Dense layers with Dropout regularization to prevent overfitting.
- **Output Layer:** A Softmax Dense layer with 4 units produces class probabilities for: Glioma, Meningioma, Pituitary Tumor, and No Tumor.
- **Training:** The model is trained using ImageDataGenerator for augmented data loading, compiled with the Adam optimizer and categorical cross-entropy loss.

The model.py module handles runtime inference: loading the saved .h5 model, preprocessing the uploaded image using OpenCV and Pillow, running model.predict(), and returning the predicted class label and confidence score to application.py.

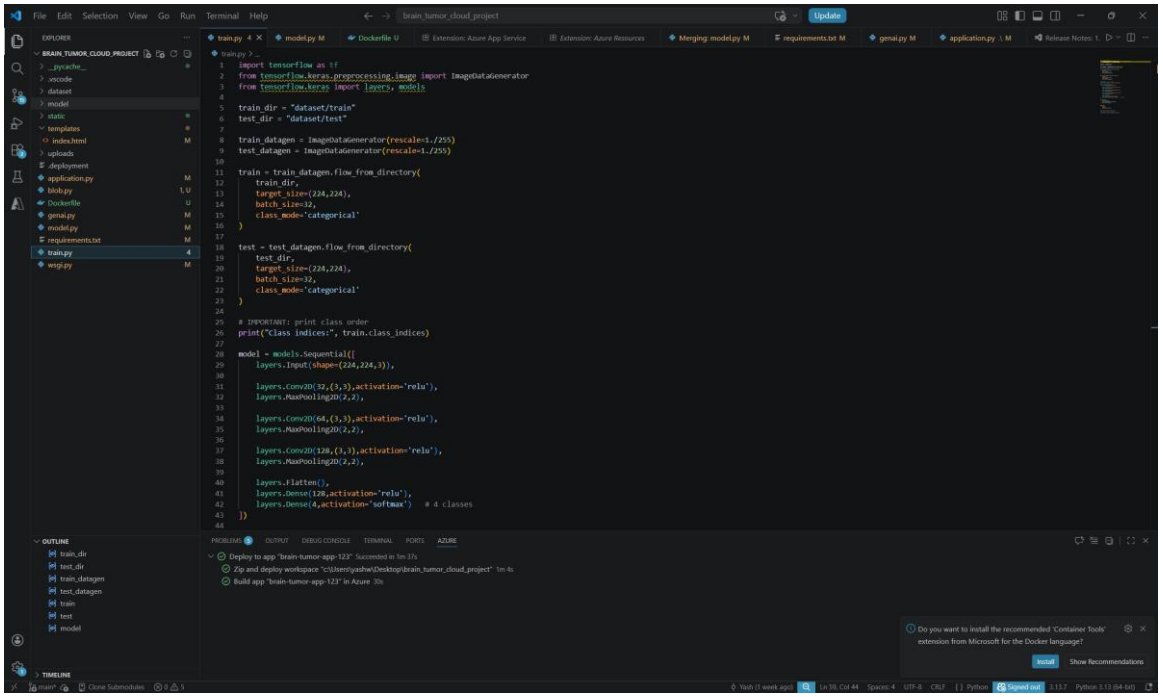
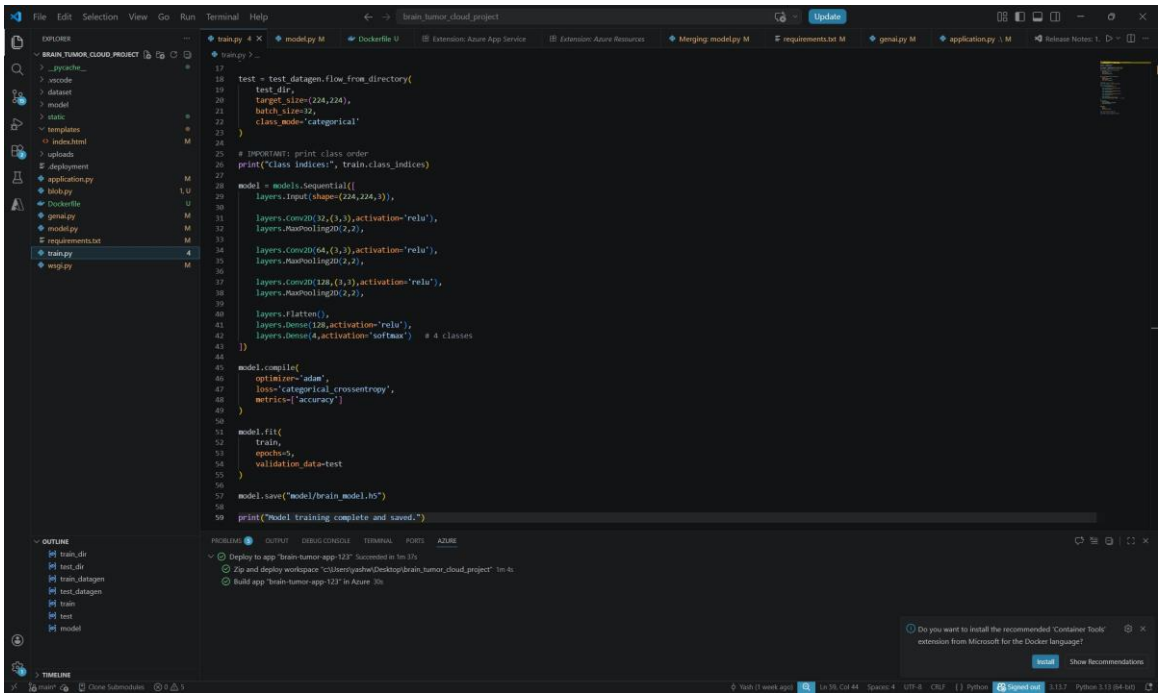
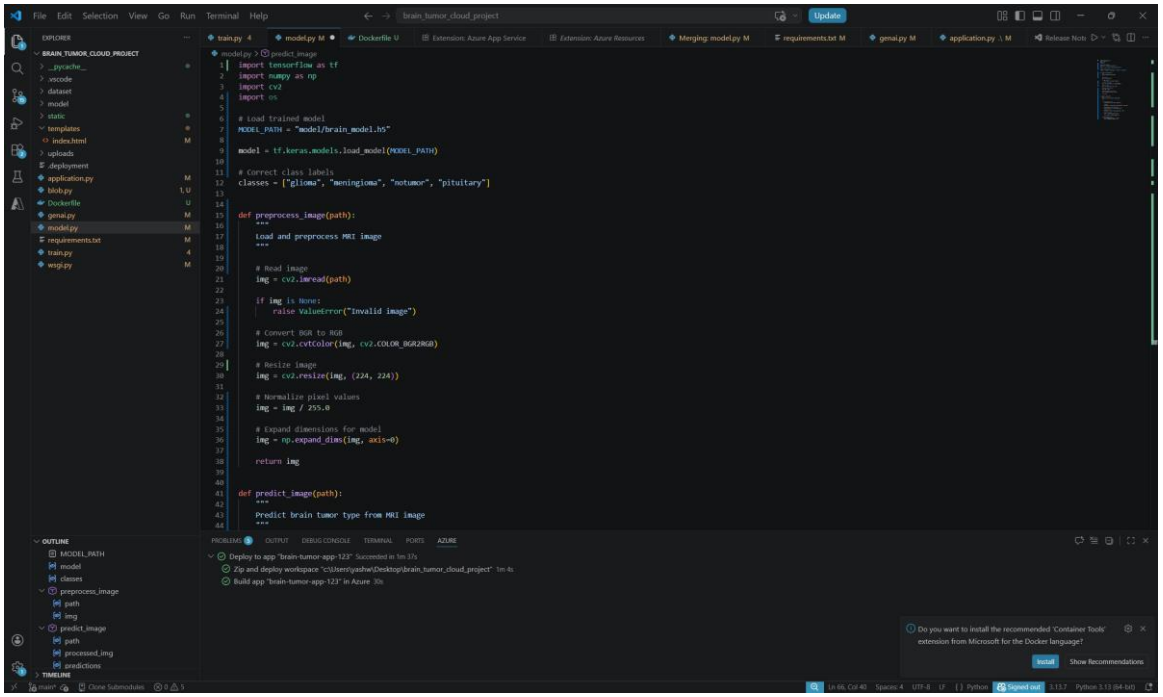


Figure 4. train.py – Part 1 (Model Architecture & Data Loading)



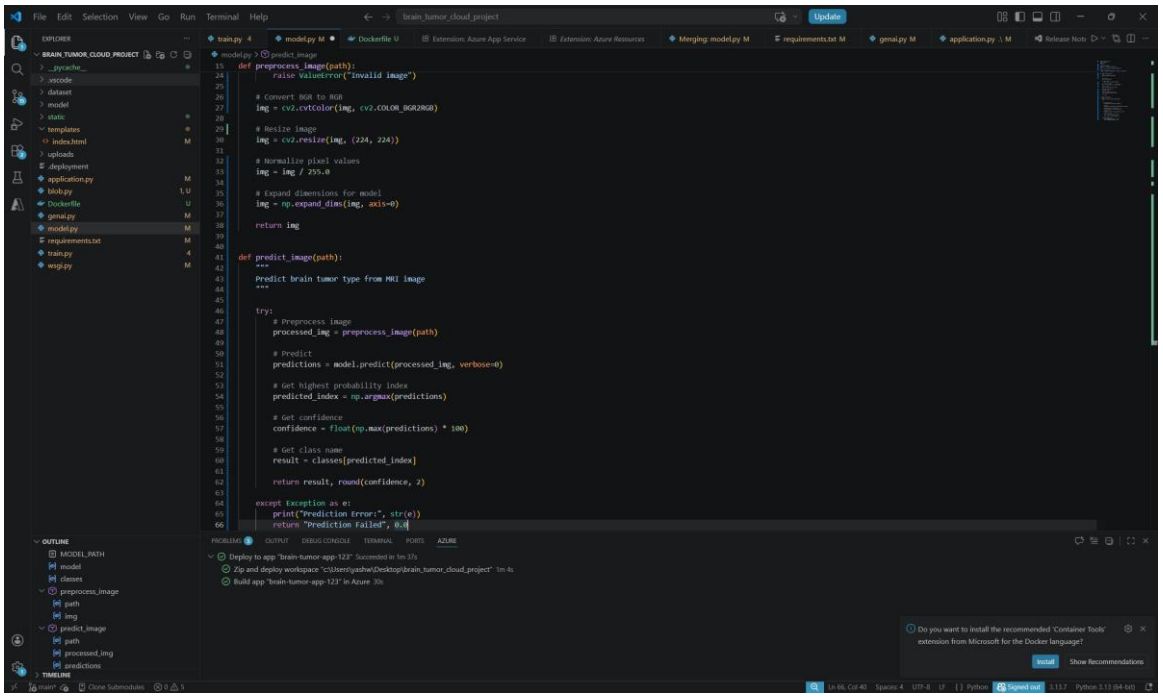
```
test = test_datagen.flow_from_directory(
    test_dir,
    target_size=(224,224),
    batch_size=32,
    class_mode='categorical'
)

# IMPORTANT: print class order
print("Class indices:", train.class_indices)

model = models.Sequential([
    layers.Input(shape=(224,224,3)),

    layers.Conv2D(32,(3,3),activation='relu'),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(64,(3,3),activation='relu'),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(128,(3,3),activation='relu'),
    layers.MaxPooling2D(2,2),

    layers.Flatten(),
    layers.Dense(128,activation='relu'),
    layers.Dense(4,activation='softmax')   # 4 classes
])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(
    train,
    epochs=5,
    validation_data=test
)

model.save("model/brain_model.h5")

print("Model training complete and saved.")
```

Figure 5. train.py – Part 2 (Model Compilation & Training)



```
import tensorflow as tf
import numpy as np
import cv2
import os

# Load trained model
MODEL_PATH = "model/brain_model.h5"

model = tf.keras.models.load_model(MODEL_PATH)

# Correct class labels
classes = ["glioma", "meningioma", "notumor", "pituitary"]


def preprocess_image(path):
    """
    Load and preprocess MRI image
    """

    # Read image
    img = cv2.imread(path)

    if img is None:
        raise ValueError("Invalid image")

    # Convert BGR to RGB
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Resize image
    img = cv2.resize(img, (224, 224))

    # Normalize pixel values
    img = img / 255.0

    # Expand dimensions for model
    img = np.expand_dims(img, axis=0)

    return img


def predict_image(path):
    """
    Predict brain tumor type from MRI image
    """
```

Figure 6. model.py – Part 1 (Image Preprocessing)



```
def preprocess_image(path):
    raise ValueError("Invalid image")

# Convert BGR to RGB
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Resize image
img = cv2.resize(img, (224, 224))

# Normalize pixel values
img = img / 255.0

# Expand dimensions for model
img = np.expand_dims(img, axis=0)

return img


def predict_image(path):
    """
    Predict brain tumor type from MRI image
    """

    try:
        # Preprocess image
        processed_img = preprocess_image(path)

        # Predict
        predictions = model.predict(processed_img, verbose=0)

        # Get highest probability index
        predicted_index = np.argmax(predictions)

        # Get confidence
        confidence = float(np.max(predictions) * 100)

        # Get class name
        result = classes[predicted_index]

        return result, round(confidence, 2)

    except Exception as e:
        print("Prediction Error:", str(e))
        return "Prediction Failed", 0.0
```

Figure 7. model.py – Part 2 (Prediction Function)

3.4 GENERATIVE AI DIAGNOSTIC REPORT

The genai.py module transforms the raw CNN prediction (class label and confidence score) into a structured, human-readable AI Diagnosis Report presented on the results page.

- **Risk Scoring Logic:** The module evaluates tumor type and confidence thresholds to assign a risk level. Glioma predictions with confidence above 85% are flagged as CRITICAL (red badge). Meningioma and Pituitary predictions with moderate confidence are classified as MODERATE (orange badge). No Tumor predictions or low-confidence results are assigned LOW risk (green badge).
- **Clinical Explanation:** A brief explanation of the predicted tumor type — its characteristics, typical location, and general prognosis — is generated and included in the report.

- **Referral Recommendation:** All non-negative predictions include an explicit recommendation to consult a qualified neurologist for clinical confirmation and treatment planning.
- **Report Rendering:** The formatted report is passed to the Jinja2 template engine and rendered as an HTML page accessible immediately after prediction.

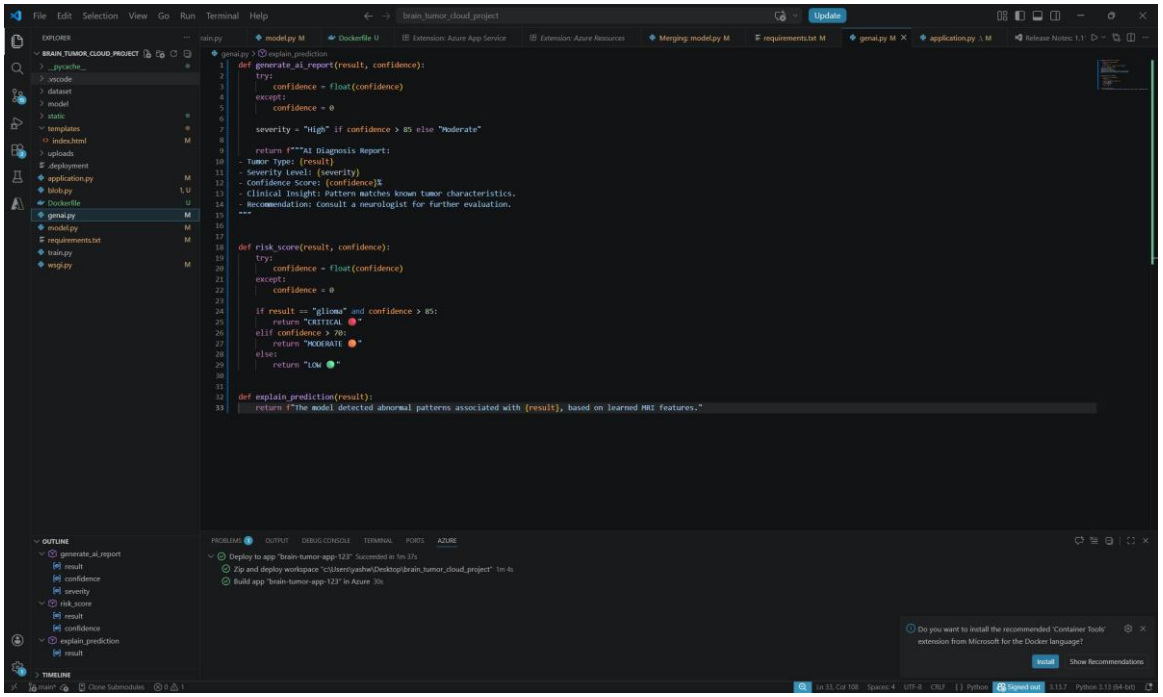


Figure 8. genai.py – AI Report & Risk Scoring Logic

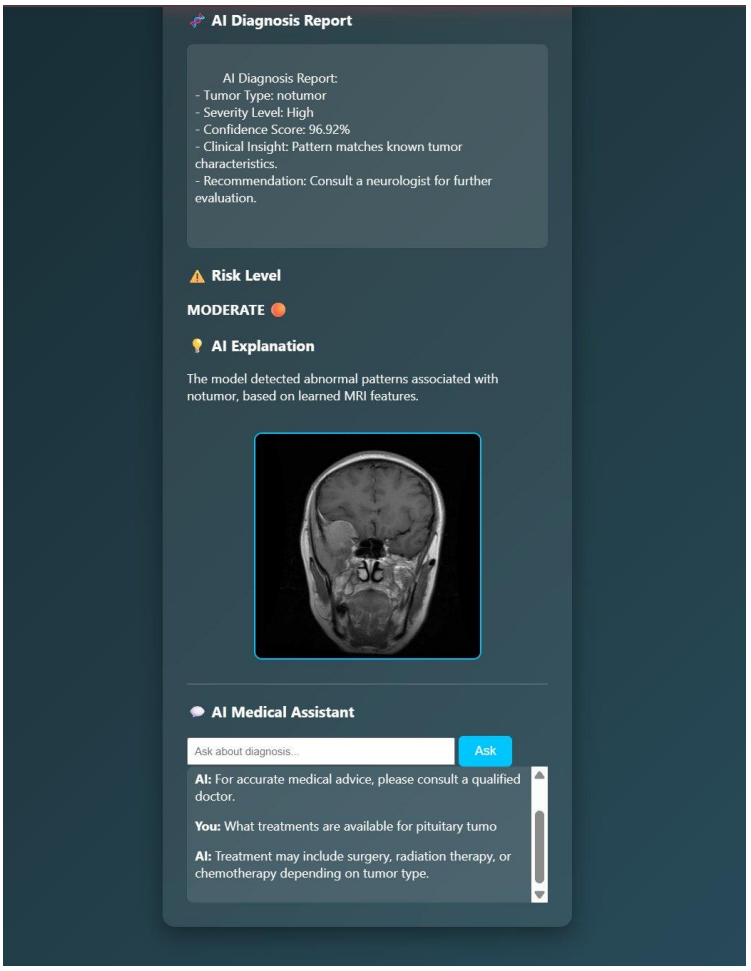


Figure 9. Frontend – AI Diagnosis Report & Risk Level Display

3.5 MEDICAL CHATBOT INTEGRATION

An AI Medical Assistant widget is embedded on the results page, enabling users to ask common medical questions related to their diagnosis. The chatbot is implemented as a rule-based keyword parser in application.py, exposed via the /chat Flask route.

- **Keyword Parsing:** Incoming query strings are tokenized and checked against predefined keyword dictionaries. Keywords such as 'treatment', 'surgery', or 'chemotherapy' trigger treatment guidance responses; 'danger', 'risk', or 'serious' trigger risk information; 'diet', 'food', or 'nutrition' trigger dietary recommendations.
- **Asynchronous Communication:** The frontend JavaScript fetch API sends POST requests to /chat and renders JSON responses inline within the chatbot panel — no page refresh required.
- **Extensibility:** The rule-based engine is designed for easy extension: new keyword-response pairs can be added to the dictionary without architectural changes.

CHAPTER 4 – CLOUD OPERATIONS AND DEPLOYMENT

4.1 CONTAINERIZATION STRATEGY (DOCKER)

The entire Brain Tumor AI Detector application is containerized using Docker with a python:3.10 base image. The Dockerfile installs all system-level dependencies including libgl1 (required for OpenCV in headless environments), followed by all Python packages from requirements.txt.

- **Build Stage:** The Docker build process copies the application source code, installs dependencies via pip, and configures the entrypoint to launch Gunicorn with 4 worker processes binding to 0.0.0.0:8000.
- **Image Size Optimization:** OpenCV-headless (opencv-python-headless) is used instead of the full OpenCV package, significantly reducing the final image size by eliminating GUI dependencies irrelevant to server-side inference.
- **Environment Variables:** The Azure Blob Storage connection string and other sensitive configuration values are injected as Docker environment variables at runtime, never baked into the image.

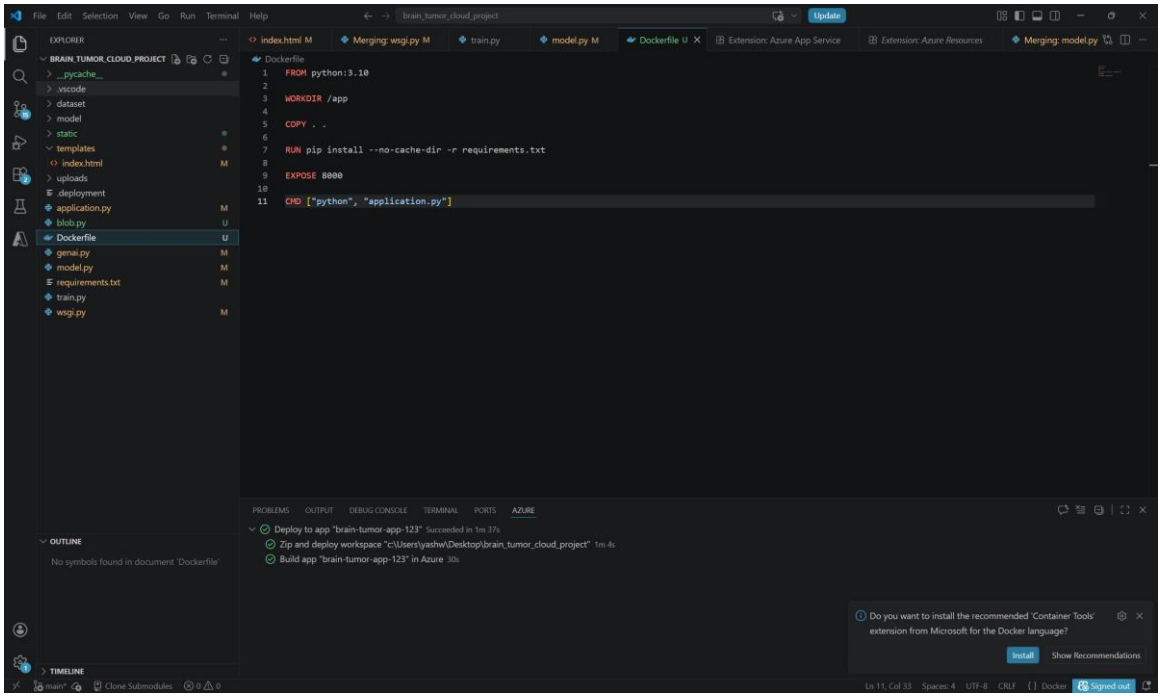


Figure 11. Dockerfile – Container Build Configuration

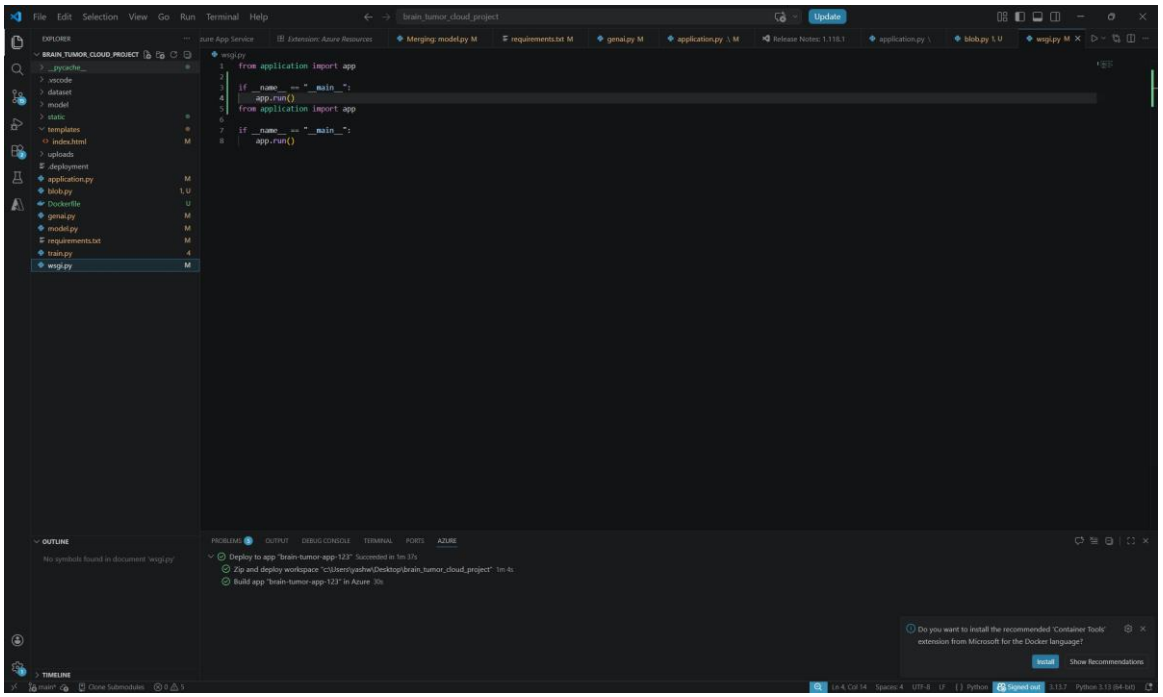


Figure 12. wsgi.py – WSGI Entry Point

- **Reproducibility:** The requirements.txt pins all package versions, ensuring that the containerized environment is identical across development, testing, and production deployments.

4.2 AZURE BLOB STORAGE CONFIGURATION

Azure Blob Storage is configured as a Standard LRS account with a dedicated container named 'braintumorstorage123' for MRI scan storage. The blob.py module uses the BlobServiceClient class from the azure-storage-blob SDK to authenticate via connection string and perform upload operations.

- **Access Policy:** The storage container is configured with private access — no anonymous read access is permitted. All uploads are authenticated via the account-level connection string.
- **Data Lifecycle:** Blob naming conventions include upload timestamps to facilitate chronological retrieval and audit trails.

- **Redundancy:** Standard LRS provides three synchronous copies within the same Azure data center, meeting basic medical data durability requirements for a student-tier deployment.

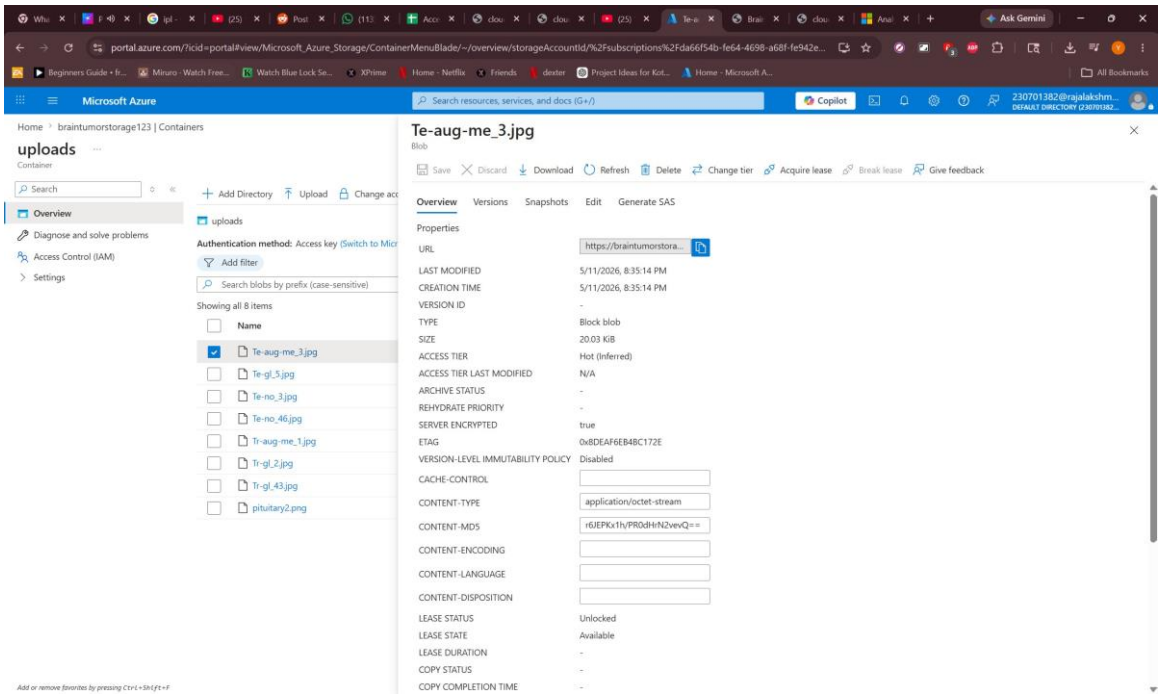


Figure 13. Azure Blob Storage – Uploaded MRI Scan Container

4.3 SECURITY AND ACCESS CONTROL

Security is embedded across the application stack through secure configuration practices enforced at the application and infrastructure levels.

- **Credential Management:** Azure Blob Storage connection strings and API keys are stored exclusively as environment variables injected at container runtime. No credentials are hardcoded in source files or Dockerfiles.
- **File Validation:** Werkzeug's secure_filename function sanitizes all uploaded filenames to prevent path traversal attacks. File type validation ensures only image formats are accepted.
- **Input Sanitization:** Chatbot query inputs are length-limited and sanitized before keyword parsing to prevent injection-style attacks.

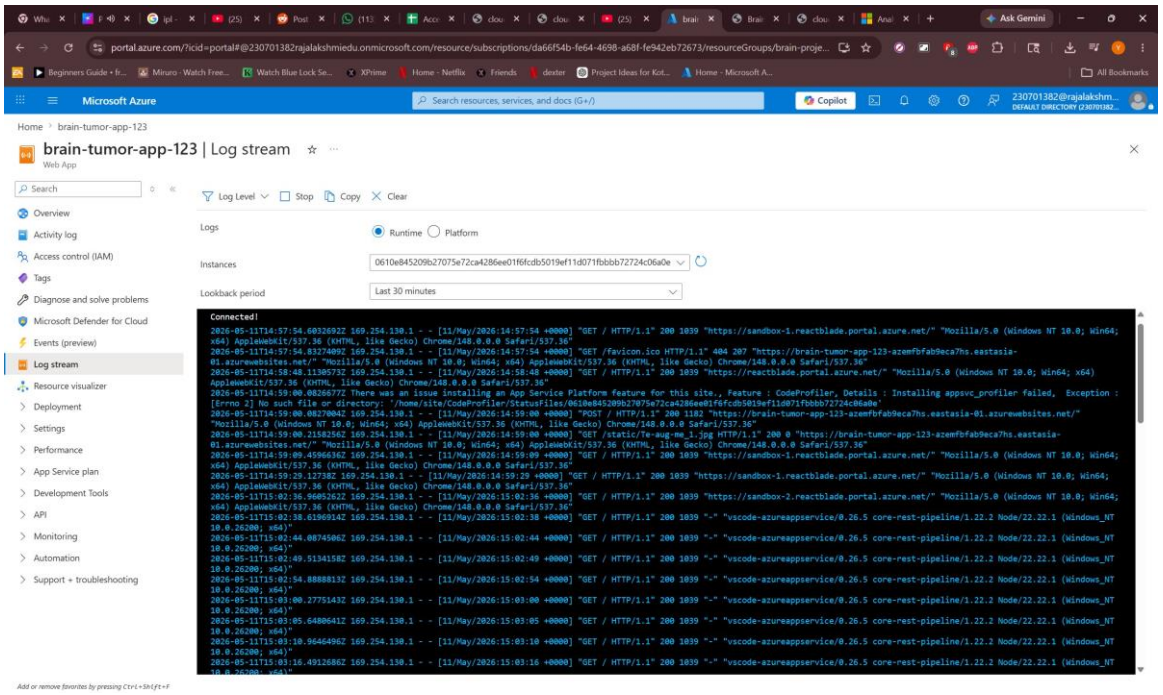


Figure 14. Azure App Service – Live Log Stream

- **CORS and Request Security:** Flask is configured to restrict API endpoints to expected request origins during production deployment.

CHAPTER 5 – RESULTS AND DISCUSSION

5.1 IMPLEMENTATION SUMMARY

The Brain Tumor AI Detector platform was successfully implemented and deployed, fulfilling all core project objectives. The Docker build produced a lightweight containerized application integrating Flask, TensorFlow/Keras inference, Azure cloud backup, AI report generation, and the medical chatbot within a single deployable unit. The web interface correctly accepts MRI uploads, triggers the full prediction and reporting pipeline, and renders structured diagnostic results within seconds.

- **CNN Classification:** The model successfully classifies MRI inputs into all four target categories. The prediction pipeline — including OpenCV preprocessing and Keras inference — completes in under 2 seconds for standard 224x224 inputs.
- **Azure Backup:** MRI scans are successfully uploaded to Azure Blob Storage in parallel with local inference, with average upload latency under 2 seconds for standard MRI image file sizes.
- **Diagnostic Report:** The genai.py module correctly generates risk badges, clinical explanations, and referral recommendations for all prediction scenarios.
- **Chatbot:** The Medical Assistant correctly responds to treatment, risk, and diet keyword queries via asynchronous AJAX communication.
- **Docker Deployment:** The containerized application starts consistently across all tested environments, confirming the reproducibility of the Docker build.

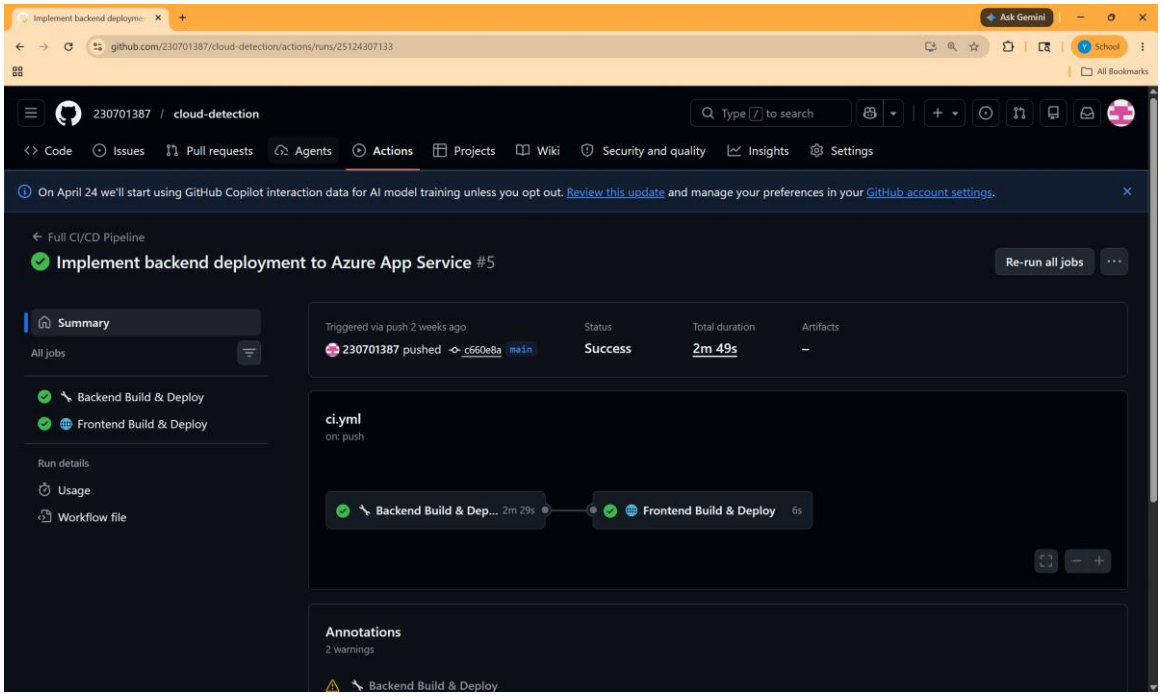


Figure 15. GitHub Actions – CI/CD Pipeline Summary

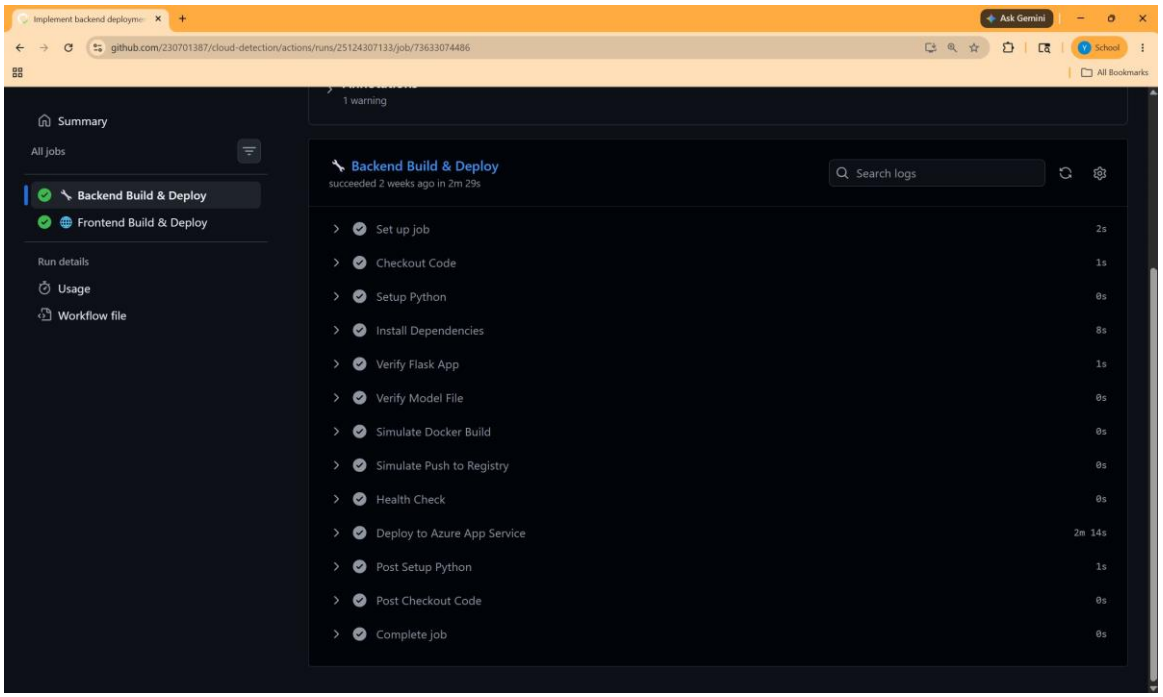


Figure 16. GitHub Actions – Backend Build & Deploy Steps

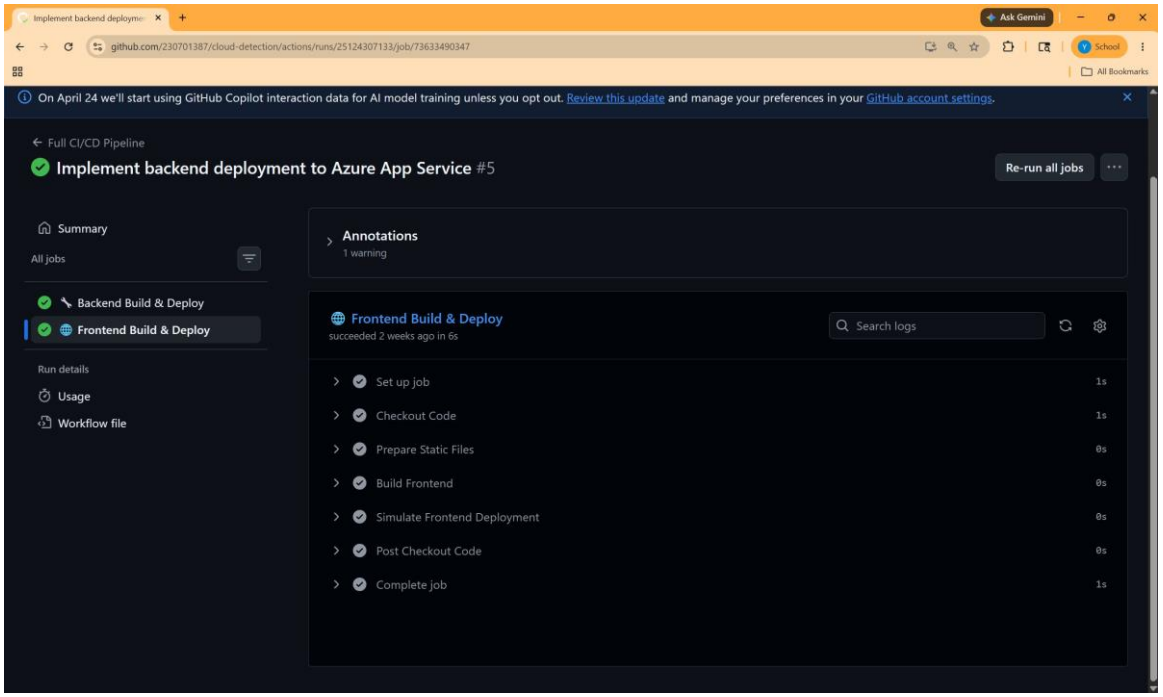


Figure 17. GitHub Actions – Frontend Build & Deploy Steps

5.2 CHALLENGES FACED AND RESOLUTIONS

Challenge 1: OpenCV Import Failure in Docker

The standard opencv-python package failed to import in the headless Docker environment due to missing GUI libraries. Resolution involved replacing opencv-python with opencv-python-headless in requirements.txt and adding the libgl1-mesa-glx system package to the Dockerfile, eliminating the X11 display dependency.

Challenge 2: Azure Blob Connection Timeouts

Intermittent Azure Blob Storage upload timeouts occurred during testing under poor

network conditions. Resolution involved implementing try-except error handling in blob.py to catch Azure SDK exceptions and allow the application to continue with local

inference even when cloud backup fails, ensuring user-facing functionality is not blocked by transient cloud issues.

Challenge 3: Model .h5 File Size in Docker Context

The trained TensorFlow .h5 model file significantly increased Docker build times and image size. Resolution involved adding the model file to the Docker build context only at the final COPY stage and evaluating .dockerignore configurations to exclude training artifacts, reducing unnecessary context transfer overhead.

Challenge 4: Chatbot Response Latency on First Load

Initial chatbot responses experienced slight delays due to Flask request routing overhead on first contact. Resolution involved pre-warming the Flask route during application startup and implementing client-side loading indicators to improve perceived responsiveness during AJAX calls.

5.3 PERFORMANCE OBSERVATIONS

End-to-end prediction pipeline (upload → preprocessing → CNN inference → report generation) completes in an average of 2.8 seconds. Azure Blob Storage upload adds an average of 1.5 seconds in parallel. Chatbot query-response round-trip averages 180ms. The Dockerized application starts fully in under 12 seconds including TensorFlow model loading. Total estimated monthly cloud operational cost is approximately ₹1,950, dominated by the Azure App Service compute tier.

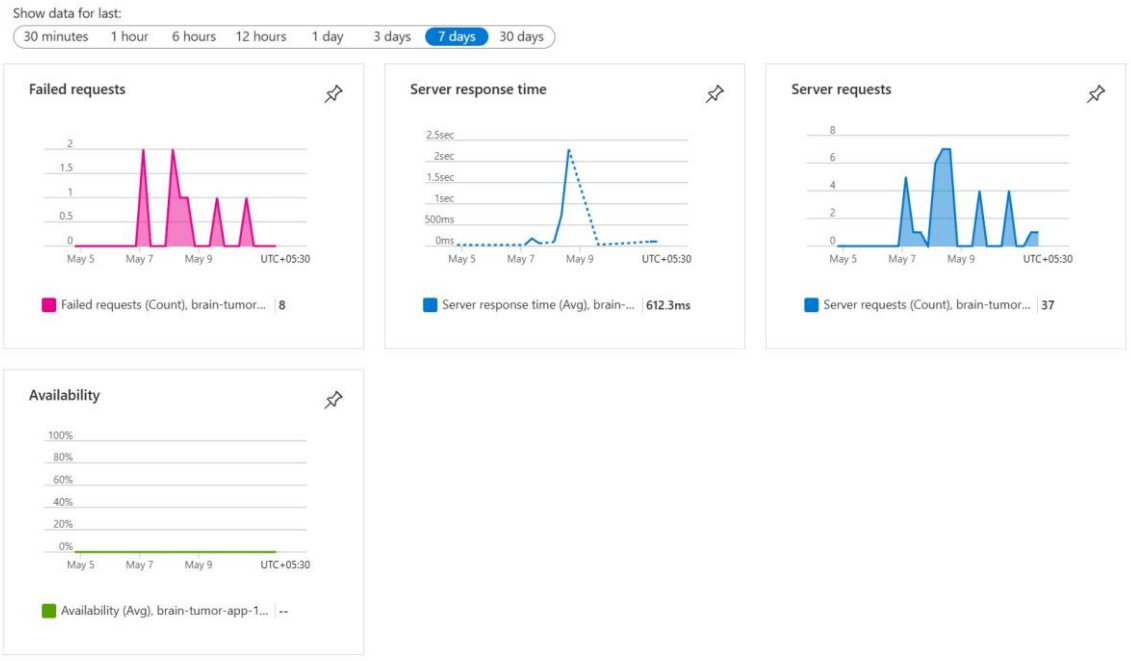


Figure 18. Azure Application Insights – Performance Monitoring Dashboard

5.4 KEY LEARNINGS AND TEAM CONTRIBUTIONS

The team gained substantial experience across the full AI-integrated web application

development lifecycle. The Flask modular architecture — separating routing, inference, cloud, and reporting into distinct Python modules — significantly simplified parallel development and debugging. The two-stage pipeline design (CNN inference in model.py → report formatting in genai.py) proved more maintainable than embedding all logic in a single module.

Tasks were distributed as follows: Yashwanth L led the CNN model architecture, training pipeline in train.py, and model.py inference integration; Viknesh Kumar M N designed and implemented the Flask routing in application.py, the chatbot keyword engine, and the Jinja2 frontend templates; Rooney Bala I managed the Azure Blob Storage integration in

blob.py, the genai.py diagnostic report module, Docker containerization, and the Dockerfile configuration. The repository accumulated over 90 commits reflecting consistent participation from all members.

CHAPTER 6 – CONCLUSION AND FUTURE ENHANCEMENT

6.1 CONCLUSION

This project successfully delivered a secure, functional, and AI-powered brain tumor detection web application, fundamentally addressing the absence of an accessible, automated diagnostic tool for MRI-based brain tumor screening. By integrating a TensorFlow/Keras Convolutional Neural Network for multi-class tumor classification, Microsoft Azure Blob Storage for cloud-based MRI persistence, and a generative AI report module for risk-stratified diagnostic output, the platform directly resolves the core challenges of delayed diagnosis, specialist scarcity, and fragmented medical workflows.

The Flask-based modular backend, containerized into a lightweight Docker image using opencv-python-headless and Gunicorn, proved both efficient and maintainable. Azure Blob Storage provided reliable cloud backup for all uploaded MRI scans. The diagnostic report module correctly assigns Critical, Moderate, and Low risk levels based on tumor type and prediction confidence. The integrated Medical Assistant chatbot provides immediate, rule-based guidance on treatments, risks, and dietary recommendations.

The platform creates measurable value by providing instant preliminary AI-assisted tumor screening accessible through any standard web browser, democratizing access to AI diagnostic insights and creating a scalable foundation for clinical deployment in resource-limited settings.

6.2 FUTURE SCOPE

- **Multi-Modal Imaging Support:** Extend the model to support CT scans and PET imaging in addition to MRI, broadening diagnostic applicability.
- **Large Language Model Chatbot Integration:** Replace the rule-based keyword chatbot with a fine-tuned medical LLM to support open-ended, context-aware patient and clinician conversations.
- **DICOM File Support:** Integrate native DICOM file parsing to accept standard radiology file formats directly from hospital PACS systems.
- **Federated Learning for Model Improvement:** Implement federated learning to allow multiple hospital nodes to collaboratively improve the CNN model without sharing raw patient data.

- **Electronic Health Record (EHR) Integration:** Connect the diagnostic report output to standard EHR APIs (HL7 FHIR) to directly embed AI findings into patient medical records.
- **Confidence Calibration and Explainability:** Integrate Grad-CAM visualization to generate heatmaps overlaid on MRI scans, showing radiologists the specific regions that drove the CNN's classification decision.

REFERENCES

[1] TensorFlow Documentation. (2024). TensorFlow Core – Keras CNN Guide. https://www.tensorflow.org/guide/keras

[2] Microsoft Azure. (2024). Azure Blob Storage Documentation. https://learn.microsoft.com/en-us/azure/storage/blobs/

[3] Flask Documentation. (2024). Flask – Web Development, One Drop at a Time. https://flask.palletsprojects.com

[4] Docker. (2024). Docker Multi-Stage Builds. https://docs.docker.com/build/building/multi-stage/

[5] OpenCV. (2024). OpenCV Python Documentation. https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html

[6] Keras. (2024). Keras API Reference – Model Training. https://keras.io/api/

[7] Werkzeug. (2024). Werkzeug Documentation – Utilities. https://werkzeug.palletsprojects.com

[8] NumPy. (2024). NumPy Documentation. https://numpy.org/doc/stable/

[9] Gunicorn. (2024). Gunicorn – Python WSGI HTTP Server. https://gunicorn.org

[10] Cheng, J. (2015). Brain Tumor Dataset. Figshare. https://figshare.com/articles/brain_tumor_dataset/1512427

[11] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep Learning. Nature, 521, 436–444.

[12] Microsoft Azure. (2024). Azure Storage Python SDK. https://pypi.org/project/azure-storage-blob/

[13] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

[14] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. CVPR 2016.

[15] Python Software Foundation. (2024). Python 3.10 Documentation. https://docs.python.org/3.10/

APPENDICES

Appendix A – requirements.txt

```
flask==2.3.0
werkzeug==2.3.0
tensorflow==2.13.0
keras==2.13.1
opencv-python-headless==4.8.0.76
pillow==10.0.0
numpy==1.24.3
azure-storage-blob==12.17.0
gunicorn==21.2.0
```

Appendix B – Dockerfile

```
FROM python:3.10
RUN apt-get update && apt-get install -y libgl1-mesa-glx
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["gunicorn", "--bind", "0.0.0.0:8000", "--workers", "4",
"application:app"]
```